ASSIGNMENT 1 – TASK 12: Re-evaluate and Adapt the Design

Bloom's Level: K5 – Evaluate | Marks: 8

The changed requirements are: (1) e-bikes have a flat parking fee; (2) delivery vans are charged at twice the car rate between 9:00 AM and 5:00 PM; (3) visitors can buy a one-day visitor pass; and (4) the facility can have 3 to 5 floors, with different slot counts on each floor. This task is worth 8 marks, split into Parts A–D at 2 marks each.

Part A – Identify changed requirements and affected classes (2 marks)

Add E-Bike and DeliveryVan vehicle categories. Add a VisitorPass with a validity date/time range. Keep ParkingFacility configurable with an int[] slotsPerFloor, so 3, 4 or 5 floors and unequal floor capacities can be represented without rewriting allocation logic. Keep rates and rules outside the facility's slot-search code.

Part B – Select an extensible design (2 marks)

Retain the stable ParkingFacility, ParkingSlot and ParkingTicket concepts. Modify the vehicle hierarchy to add E-Bike and DeliveryVan classes, and use a PricingRule interface with separate implementations for flat e-bike pricing and time-window delivery-van pricing. Model the one-day pass as a separate object that is checked at entry/fee calculation. This is better than a long if-else chain because each new rule is isolated.

Part C – Working Java code and tests (2 marks)

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.LocalTime;

interface PricingRule {
    double fee(LocalDateTime entry, LocalDateTime exit, double carRate);
}

class CarRule implements PricingRule {
    public double fee(LocalDateTime entry, LocalDateTime exit, double carRate) {
        long minutes = Duration.between(entry, exit).toMinutes();
        if (minutes < 0) throw new IllegalArgumentException("Invalid time range");
        return (minutes / 60.0) * carRate;
    }
}

class EBikeRule implements PricingRule {
    private final double flatFee;
    EBikeRule(double flatFee) { this.flatFee = flatFee; }

    public double fee(LocalDateTime entry, LocalDateTime exit, double carRate) {
        if (exit.isBefore(entry)) throw new IllegalArgumentException("Invalid time range");
        return flatFee; // flat fee regardless of parking duration
    }
}

class DeliveryVanRule implements PricingRule {
    public double fee(LocalDateTime entry, LocalDateTime exit, double carRate) {
        if (exit.isBefore(entry)) throw new IllegalArgumentException("Invalid time range");

        // Base car-rate charge for the whole duration.
        long totalMinutes = Duration.between(entry, exit).toMinutes();
        double fee = (totalMinutes / 60.0) * carRate;

        // Add one extra car-rate charge only for minutes overlapping 09:00–17:00.
        LocalDateTime day = entry.toLocalDate().atStartOfDay();
        LocalDateTime lastDay = exit.toLocalDate().atStartOfDay();
        while (!day.isAfter(lastDay)) {
            LocalDateTime windowStart = day.toLocalDate().atTime(LocalTime.of(9, 0));
            LocalDateTime windowEnd = day.toLocalDate().atTime(LocalTime.of(17, 0));
            LocalDateTime overlapStart = entry.isAfter(windowStart) ? entry : windowStart;
```

```
            LocalDateTime overlapEnd = exit.isBefore(windowEnd) ? exit : windowEnd;

            if (overlapEnd.isAfter(overlapStart)) {
                long overlapMinutes = Duration.between(overlapStart, overlapEnd).toMinutes();
                fee += (overlapMinutes / 60.0) * carRate;
            }
            day = day.plusDays(1);
        }
        return fee;
    }
}

class VisitorPass {
    private final LocalDateTime validFrom;
    private final LocalDateTime validUntil;

    VisitorPass(LocalDateTime validFrom, LocalDateTime validUntil) {
        if (validUntil.isBefore(validFrom)) {
            throw new IllegalArgumentException("Invalid pass period");
        }
        this.validFrom = validFrom;
        this.validUntil = validUntil;
    }

    boolean isValid(LocalDateTime entry, LocalDateTime exit) {
        return !entry.isBefore(validFrom) && !exit.isAfter(validUntil);
    }
}

public class Task12Demo {
    public static void main(String[] args) {
        LocalDateTime entry = LocalDateTime.of(2026, 10, 25, 10, 0);
        LocalDateTime exit = entry.plusHours(2);

        PricingRule car = new CarRule();
        PricingRule ebike = new EBikeRule(15.0);
        PricingRule van = new DeliveryVanRule();

        System.out.println(car.fee(entry, exit, 30));
        System.out.println(ebike.fee(entry, exit, 30));
        System.out.println(van.fee(entry, exit, 30));

        VisitorPass pass = new VisitorPass(
            entry.toLocalDate().atStartOfDay(),
            entry.toLocalDate().atTime(23, 59, 59));
        System.out.println(pass.isValid(entry, exit));

        int[] slotsPerFloor = {20, 25, 30}; // unequal capacities, 3 floors
        System.out.println(slotsPerFloor.length);
    }
}
```

Test results

Test	Expected output	Actual output
Car, 120 minutes, ₹30/hour	60.0	60.0 – PASS
E-bike, 120 minutes, flat fee ₹15	15.0	15.0 – PASS
Delivery van, 120 minutes starting 10:00, ₹30 car rate	120.0 (double rate in 09:00–17:00)	120.0 – PASS
One-day pass valid for the entry and exit	true	true – PASS
Unequal floor configuration {20,25,30}	3 floors	3 – PASS

The delivery-van code applies the normal car rate to the full parking duration, then adds a second car-rate charge only for the exact minutes overlapping 09:00–17:00. Therefore a van parked 08:00–10:00 is charged one hour at the normal rate plus one hour at double rate. The example uses prorated minutes to make boundary handling precise; the campus should confirm whether its official policy instead rounds to started hours.

Part D – Evaluate extensibility and complexity (2 marks)

The revised design should be retained with targeted modifications, not fully redesigned. PricingRule composition lets a new pricing rule be added without editing ParkingFacility or existing vehicle classes. Allocating a slot with a simple scan is O(N) in the worst case for N total slots; release by known floor/slot is O(1). Fee calculation is O(1) for the fixed rules above. The floor array and slots require O(N) space. The VisitorPass adds O(1) data per pass. The flat fee, time window and one-day pass rules are independently testable.

Facility capacity is configurable by passing an array with 3 to 5 values, for example {20, 25, 30} or {50, 40, 60, 35, 45}. The allocation code must use each row's actual length, not assume equal floor sizes.

Overall conclusion

The recommended solution separates vehicles, slots, tickets, user passes and pricing rules. It validates entry/exit operations, preserves unique ticket numbers, uses suitable Java collections and keeps new pricing rules isolated. The design can accommodate the stated requirements with focused additions rather than rewriting stable classes.